Démonstrateur d'attaques contre un ML-NIDS

Réunion de début de projet

Projet VAP SSR 2026–2027

Encadrant : Gregory Blanc

6 octobre 2026

Notre compréhension du projet

Objectif

Un démonstrateur pédagogique pour comprendre les attaques contre un ML-NIDS et observer leurs effets.

Livrables attendus

- Bref état de l'art des attaques contre les ML-NIDS.
- Logiciel de démonstration en trois parties :
  1. Choix de l'environnement réseau, façon Packet Tracer.
  2. Choix du ML-NIDS.
  3. Choix de l'attaque effectuée.

Une vue pour comprendre l'attaque

Visualiser son déroulement en direct ou avec une comparaison avant/après.

JIRA

ML-NIDS et attaques concrètes identifiés

Modèles ciblés	**Manipulation adversariale**	**Papiers sources**
KitNET / autoencodeurs, Isolation Forest	Modifier les délais ; ajouter des paquets.	Han (2021), Sharon (2022).
MLP	Modifier le comportement TCP à partir des caractéristiques importantes.	Okada (2025).
MLP, CNN, ResCNN, LSTM, attention	Perturbations transférables ou guidées par des requêtes au détecteur.	Mao (2025), Zhang (2022).
E-GraphSAGE, Line-GraphSAGE	Ajouter des communications pour modifier le graphe observé.	Venturi (2024).
SVM, Random Forest, MLP	Injecter des données mal étiquetées ou modifier les labels d'entraînement.	Venkatesan (2021), Das (2025).

Ces couples modèle–attaque sont des pistes bibliographiques : leur intégration dans le démonstrateur reste à étudier.

Changer le rythme du trafic pour passer inaperçu

Modèles concernés : KitNET et d'autres détecteurs d'anomalies.

Principe

Le détecteur observe la façon dont les paquets circulent. L'attaquant change ce rythme pour que son trafic ressemble davantage à une activité normale.

- **TrafficManipulator :** espacer les envois et ajouter des paquets.
- **TANTRA :** modifier les délais entre les paquets sans changer leur contenu.

Idée de visualisation : une chronologie des paquets avant / après, accompagnée du score de détection.

Papiers : Han et al. (2021), TrafficManipulator ; Sharon et al. (2022), TANTRA.

Repérer ce que regarde le détecteur, puis l'adapter

Modèle concerné : un réseau de neurones classique (MLP).

Principe

Chercher quels aspects d'une communication influencent la décision du modèle, puis les modifier pour rendre l'attaque moins visible.

- Exemple étudié : une attaque qui essaie de nombreux mots de passe.
- Changer la manière de se connecter, par exemple en ouvrant une connexion différente à chaque tentative.
- L'objectif reste le même, mais la communication vue par le détecteur change.

Idée de visualisation : les connexions et les alertes avant / après.

Papier : Okada et al. (2025), Xai-driven black-box adversarial attacks on network intrusion detectors.

Chercher une modification qui trompe un réseau de neurones

Modèles concernés : plusieurs types de réseaux de neurones.

Deux façons de chercher une évasion

- **Sur un autre modèle :** préparer l'attaque sur un modèle accessible, puis essayer de tromper le détecteur cible avec la même modification.
- **Par essais successifs :** tester des modifications et utiliser les réponses du détecteur pour ajuster l'attaque.

Idée de visualisation : suivre les essais et voir à quel moment une activité malveillante est classée comme normale.

Papiers : Mao et al. (2025), Exploring transferable adversarial attacks...
Zhang et al. (2022), Adversarial Attacks Against Deep Learning-Based NIDS...

Ajouter des communications pour brouiller le contexte

Modèles concernés : réseaux de neurones sur graphes.

Principe

Ces modèles regardent aussi qui communique avec qui. L'attaquant ajoute des échanges normaux autour d'une machine compromise pour modifier le contexte observé par le détecteur.

- La communication malveillante reste présente.
- De nouvelles communications modifient la représentation du réseau.
- Le modèle peut alors moins bien reconnaître l'activité malveillante.

Idée de visualisation : les machines et leurs liens avant / après l'ajout des communications.

Papier : Venturi et al. (2024), Problem space structural adversarial attacks for NIDS based on Graph Neural Networks.

Faire apprendre de mauvais exemples au détecteur

Modèles concernés : SVM, forêts d'arbres et réseaux de neurones classiques.

Principe : l'empoisonnement

L'attaquant fausse une partie des exemples utilisés pour apprendre. Le détecteur risque alors d'apprendre de mauvaises règles de décision.

- Ajouter des exemples trompeurs dans les données d'entraînement.
- Ou changer leur étiquette « normal » / « malveillant ».
- L'effet apparaît après l'entraînement : il faut pouvoir agir sur ses données.

Idée de visualisation : comparer les erreurs du détecteur entraîné avec des données propres puis avec des données empoisonnées.

Papiers : Venkatesan et al. (2021), Poisoning Attacks and Data Sanitization Mitigations...
Das et al. (2025), Poisoning the Well...

Récapitulatif : ce qu'il reste à approfondir

Étudier les attaques trouvées

Comprendre leur fonctionnement précis et leurs conditions d'application.

- **Vérifier la faisabilité :** quels modèles, données et codes peut-on utiliser dans notre démonstrateur ?
- **Tester les scénarios :** l'attaque trompe-t-elle le détecteur tout en conservant son effet sur le réseau ?
- **Étoffer la sélection :** chercher d'autres attaques pour varier les mécanismes et les modèles présentés.
- **Choisir les visualisations :** rendre le principe et le résultat de chaque attaque compréhensibles.

Suite du projet : les jalons prévus

Jalon actuel

État de l'art des attaques contre les ML-NIDS : organiser les références et rédiger une synthèse comparative.

↓

Clarification de l'utilisation de Concap

Préciser son rôle et son utilisation dans le projet.

En parallèle : maquette de l'architecture logicielle

Représenter les trois parties du logiciel et leurs interactions.

Discussion avec l'encadrant

1. Le corpus réuni couvre-t-il les principales attentes pour l'état de l'art ?
2. Quels articles ou quelles familles d'attaques approfondir en priorité ?
3. Quel niveau de détail et quel format attendre pour le bref état de l'art ?